SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

Dr. M. Anoop

QUESTIONS: 10 **Total Marks:50**

Annexure A

Scenario based

Code of Conduct:

I ___ATHIN SIVA.S___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

Given: P1 – Payment gateway transactions (High priority), P2 – Seat availability checks (Medium priority), P3 – Ticket confirmation and printing (Low priority). Burst times range from 2 ms to 20 ms.

1. Most Suitable Scheduling Algorithm

The most suitable algorithm is Priority Scheduling. Since the processes already have different importance levels, Priority Scheduling ensures that critical payment transactions are handled before less important operations.

Priority order: P1 → P2 → P3

2. Why Priority Scheduling is Suitable

Payment transactions are the most critical and receive the highest priority.

It provides faster response for important processes.

It reduces waiting time for payment transactions.

It handles real-time critical operations effectively.

Lower-priority tasks execute after critical tasks.

3. Sample Execution Order

Process	Task	Burst Time	Priority
P1	Payment	4 ms	High
P2	Seat Check	6 ms	Medium
P3	Confirmation	10 ms	Low

Execution: P1 → P2 → P3

Gantt Chart: 0 | P1 | 4 | P2 | 10 | P3 | 20

Waiting times: P1 = 0 ms, P2 = 4 ms, P3 = 10 ms.

Average Waiting Time = (0 + 4 + 10) / 3 = 4.67 ms.

4. Drawback

The major drawback is starvation. If high-priority payment transactions continuously arrive, low-priority ticket confirmation processes may wait for a long time. This can be reduced using aging, where the priority of a waiting process gradually increases.

Conclusion: Priority Scheduling is suitable because payment transactions can be executed quickly according to their importance. Aging should be used to prevent starvation.

Question 2 – Deadlock in a Banking Transaction Server

Given: T1 holds A and requests B; T2 holds B and requests C; T3 holds C and requests A.

1. Resource Allocation Graph

A → T1 → B → T2 → C → T3 → A

A → T1 means T1 holds A.

T1 → B means T1 requests B.

B → T2 means T2 holds B.

T2 → C means T2 requests C.

C → T3 means T3 holds C.

T3 → A means T3 requests A.

2. Deadlock Condition

Yes, the system is deadlocked. There is a circular wait: T1 → T2 → T3 → T1.

Mutual Exclusion – Resources can be used by only one transaction at a time.

Hold and Wait – Each transaction holds one resource while waiting for another.

No Preemption – Resources cannot be forcibly removed.

Circular Wait – T1 waits for T2, T2 waits for T3, and T3 waits for T1.

3. Technique to Fix the Problem

Use Deadlock Prevention through Resource Ordering. Assign a global order A < B < C. Transactions must request resources only in this order. This eliminates the circular wait condition.

4. Redesigned Locking Order

Use the order A → B → C. For example, T1 requests A then B, T2 requests B then C, and T3 must request A before C instead of C before A. Therefore, the circular dependency cannot form.

Conclusion: The original system is deadlocked because a circular wait exists among T1, T2 and T3. A fixed global resource ordering prevents circular waiting and avoids deadlock.

Question 3 – Mobile OS App Switching Lag

Given: The system uses Round Robin Scheduling with a time quantum of 20 ms and 20 background applications are active.

1. Why CPU Remains Idle Despite Many Active Processes

Many background apps may be in the waiting or blocked state rather than the ready state. They may be waiting for network responses, file I/O, user input, database operations, or device resources. Therefore, even though 20 apps are active, they may not require CPU execution continuously.

2. Impact of 20 ms Quantum

A 20 ms quantum can be relatively large for an interactive mobile environment. When switching apps, the foreground application may need to wait while another process completes its time quantum. With many processes, this increases response delay. A large quantum makes Round Robin behave more like FCFS, while a very small quantum causes excessive context switching.

3. Better Scheduling Method

A better method is Multilevel Feedback Queue (MLFQ). It assigns higher priority to interactive foreground applications and lower priority to CPU-intensive background processes. Another option is reducing the Round Robin quantum to around 5–10 ms for interactive processes.

4. How the Change Reduces Switching Lag

When the user opens an app, the foreground app receives higher priority and gets CPU time quickly. The user interface loads faster while background applications continue at lower priorities. This reduces switching lag without completely stopping background work.

Conclusion: The lag occurs because Round Robin treats processes relatively equally and the 20 ms quantum can delay interactive processes. MLFQ with priority for foreground apps improves responsiveness.

Question 4 – Deadlock Risk in an Operating System Update

Resources: R1 = Disk I/O, R2 = Kernel lock, R3 = Configuration lock. Current state: M1 holds R1 and waits for R2; M2 holds R2 and waits for R3; M3 holds R3 and waits for R1.

1. Identify Whether Deadlock Can Occur

Yes, this sequence can cause a deadlock because a circular dependency exists among the three modules.

2. Exact Circular Wait Condition

M1 waits for R2 held by M2. M2 waits for R3 held by M3. M3 waits for R1 held by M1. Therefore, the cycle is M1 → M2 → M3 → M1, and no module can continue.

3. Banker's Algorithm Style Solution

Before allocating resources, the OS should check whether granting a request leaves the system in a safe state. A possible resource ordering is R1 < R2 < R3. If a resource request would create an unsafe state, the OS should delay the request until the required resources become available.

A safe execution sequence can be: M1 executes and releases resources → M2 executes and releases resources → M3 executes and releases resources.

4. OS Designer Solution

The OS designer can implement consistent lock ordering so that all modules request resources in the predefined order R1 → R2 → R3.

Use lock timeouts.

Use a try-lock mechanism.

Use deadlock detection.

Use resource preemption where possible.

Minimize lock holding time.

Conclusion: The modules can deadlock because they form a circular resource dependency. Banker's Algorithm or a fixed resource order R1 → R2 → R3 prevents unsafe allocation.

Question 5 – Cloud Server CPU Utilization Analysis

Given: Each VM spends 80% of its time waiting for I/O, only 20% on CPU, and the server runs 7 VMs. Formula: $U = 1 - p^n$, where p is the I/O wait probability and n is the number of VMs.

1. Calculate CPU Utilization

p = 0.8 and n = 7

$U = 1 - (0.8)^7$

$(0.8)^7 = 0.2097152$

$U = 1 - 0.2097152 = 0.7902848$

CPU Utilization = 79.03%

2. Interpretation

The server CPU is utilized approximately 79% of the time. Since the VMs spend 80% of their time waiting for I/O, the workload is strongly I/O-bound. The CPU has approximately 20.97% idle capacity, so it is not overloaded.

3. Effect of Increasing or Decreasing VMs

Increasing the number of VMs reduces the probability that all VMs are waiting for I/O at the same time, so CPU utilization increases.

For 10 VMs: $U = 1 - (0.8)^{10} = 1 - 0.10737 = 89.26\%$.

Thus, increasing VMs from 7 to 10 increases utilization from about 79% to 89%. However, too many VMs may cause memory pressure, context-switching overhead, I/O bottlenecks and resource contention. Reducing the number of VMs decreases CPU utilization.

4. OS-Level Strategy

A good strategy is dynamic VM scheduling and load balancing.

Monitor CPU and I/O utilization.

Schedule another ready VM when one VM waits for I/O.

Balance workloads between CPU cores.

Adjust the number of active VMs.

Use asynchronous I/O where possible.

Reduce unnecessary context switching.

Conclusion: For seven VMs with an 80% I/O wait probability, CPU utilization is approximately 79.03%. Increasing VMs can improve CPU utilization, but excessive VMs can create memory and I/O bottlenecks.

bottlenecks.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand